

Coreference Resolution and Entity Linking

and even Stigand, the patriotic archbishop of Canterbury, found it advisable—”

‘Found WHAT?’ said the Duck.

‘Found IT,’ the Mouse replied rather crossly: ‘of course you know what “it” means.’

‘I know what “it” means well enough, when I find a thing,’ said the Duck: ‘it’s generally a frog or a worm. The question is, what did the archbishop find?’

Lewis Carroll, *Alice in Wonderland*

An important component of language processing is knowing *who* is being talked about in a text. Consider the following passage:

(24.1) Victoria Chen, CFO of Megabucks Banking, saw her pay jump to \$2.3 million, as the 38-year-old became the company’s president. It is widely known that she came to Megabucks from rival Lotsabucks.

mention

referent

corefer

Each of the underlined phrases in this passage is used by the writer to refer to a person named Victoria Chen. We call linguistic expressions like *her* or *Victoria Chen* **mentions** or **referring expressions**, and the discourse entity that is referred to (Victoria Chen) the **referent**. (To distinguish between referring expressions and their referents, we italicize the former.)¹ Two or more referring expressions that are used to refer to the same discourse entity are said to **corefer**; thus, *Victoria Chen* and *she* corefer in (24.1).

Coreference is an important component of natural language processing. A dialogue system that has just told the user “*There is a 2pm flight on United and a 4pm one on Cathay Pacific*” must know which flight the user means by “*I’ll take the second one*”. A question answering system that uses Wikipedia to answer a question about Marie Curie must know who *she* was in the sentence “*She was born in Warsaw*”. And a machine translation system translating from a language like Spanish, in which pronouns can be dropped, must use coreference from the previous sentence to decide whether the Spanish sentence “*“Me encanta el conocimiento”, dice.*” should be translated as “*“I love knowledge”, he says*”, or “*“I love knowledge”, she says*”. Indeed, this example comes from an actual news article in *El País* about a female professor and was mistranslated as “he” in machine translation because of inaccurate coreference resolution (Schiebinger, 2013).

discourse
model

evoked

accessed

Natural language processing systems (and humans) interpret linguistic expressions with respect to a **discourse model** (Karttunen, 1969). A discourse model (Fig. 24.1) is a mental model that the understander builds incrementally when interpreting a text, containing representations of the entities referred to in the text, as well as properties of the entities and relations among them. When a referent is first mentioned in a discourse, we say that a representation for it is **evoked** into the model. Upon subsequent mention, this representation is **accessed** from the model.

¹ As a convenient shorthand, we sometimes speak of a referring expression referring to a referent, e.g., saying that *she* refers to Victoria Chen. However, the reader should keep in mind that what we really mean is that the speaker is performing the act of referring to Victoria Chen by uttering *she*.

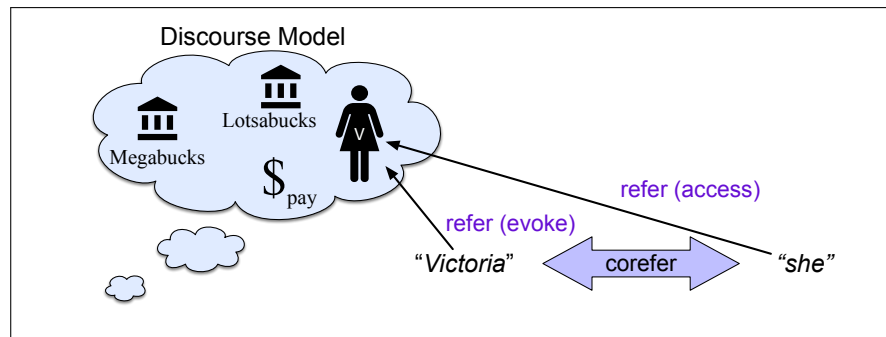


Figure 24.1 How mentions evoke and access discourse entities in a discourse model.

Reference in a text to an entity that has been previously introduced into the discourse is called **anaphora**, and the referring expression used is said to be an **anaphor**, or anaphoric.² In passage (24.1), the pronouns *she* and *her* and the definite NP *the 38-year-old* are therefore anaphoric. The anaphor corefers with a prior mention (in this case *Victoria Chen*) that is called the **antecedent**. Not every referring expression is an antecedent. An entity that has only a single mention in a text (like *Lotsabucks* in (24.1)) is called a **singleton**.

In this chapter we focus on the task of **coreference resolution**. Coreference resolution is the task of determining whether two mentions *corefer*, by which we mean they refer to the same entity in the discourse model (the same *discourse entity*). The set of coreferring expressions is often called a **coreference chain** or a **cluster**. For example, in processing (24.1), a coreference resolution algorithm would need to find at least four coreference chains, corresponding to the four entities in the discourse model in Fig. 24.1.

1. {*Victoria Chen*, *her*, *the 38-year-old*, *She*}
2. {*Megabucks Banking*, *the company*, *Megabucks*}
3. {*her pay*}
4. {*Lotsabucks*}

Note that mentions can be nested; for example the mention *her* is syntactically part of another mention, *her pay*, referring to a completely different discourse entity.

Coreference resolution thus comprises two tasks (although they are often performed jointly): (1) identifying the mentions, and (2) clustering them into coreference chains/discourse entities.

We said that two mentions corefered if they are associated with the same *discourse entity*. But often we'd like to go further, deciding which real world entity is associated with this discourse entity. For example, the mention *Washington* might refer to the US state, or the capital city, or the person George Washington; the interpretation of the sentence will of course be very different for each of these. The task of **entity linking** (Ji and Grishman, 2011) or *entity resolution* is the task of mapping a discourse entity to some real-world individual.³ We usually operationalize entity

² We will follow the common NLP usage of *anaphor* to mean any mention that has an antecedent, rather than the more narrow usage to mean only mentions (like pronouns) whose interpretation depends on the antecedent (under the narrower interpretation, repeated names are not anaphors).

³ Computational linguistics/NLP thus differs in its use of the term *reference* from the field of formal semantics, which uses the words *reference* and *coreference* to describe the relation between a mention and a real-world entity. By contrast, we follow the functional linguistics tradition in which a mention *refers* to a *discourse entity* (Webber, 1978) and the relation between a discourse entity and the real world individual requires an additional step of *linking*.

linking or resolution by mapping to an *ontology*: a list of entities in the world, like a gazeteer (Appendix H). Perhaps the most common ontology used for this task is Wikipedia; each Wikipedia page acts as the unique id for a particular entity. Thus the entity linking task of **wikification** (Mihalcea and Csomai, 2007) is the task of deciding which Wikipedia page corresponding to an individual is being referred to by a mention. But entity linking can be done with any ontology; for example if we have an ontology of genes, we can link mentions of genes in text to the disambiguated gene name in the ontology.

In the next sections we introduce the task of coreference resolution in more detail, and survey a variety of architectures for resolution. We also introduce two architectures for the task of entity linking.

Before turning to algorithms, however, we mention some important tasks we will only touch on briefly at the end of this chapter. First are the famous Winograd Schema problems (so-called because they were first pointed out by Terry Winograd in his dissertation). These entity coreference resolution problems are designed to be too difficult to be solved by the resolution methods we describe in this chapter, and the kind of real-world knowledge they require has made them a kind of challenge task for natural language processing. For example, consider the task of determining the correct antecedent of the pronoun *they* in the following example:

- (24.2) The city council denied the demonstrators a permit because
- a. they feared violence.
 - b. they advocated violence.

Determining the correct antecedent for the pronoun *they* requires understanding that the second clause is intended as an explanation of the first clause, and also that city councils are perhaps more likely than demonstrators to fear violence and that demonstrators might be more likely to advocate violence. Solving Winograd Schema problems requires finding way to represent or discover the necessary real world knowledge.

event
coreference

A problem we won't discuss in this chapter is the related task of **event coreference**, deciding whether two event mentions (such as the *buy* and the *acquisition* in these two sentences from the ECB+ corpus) refer to the same event:

- (24.3) AMD agreed to [**buy**] Markham, Ontario-based ATI for around \$5.4 billion in cash and stock, the companies announced Monday.
- (24.4) The [**acquisition**] would turn AMD into one of the world's largest providers of graphics chips.

Event mentions are much harder to detect than entity mentions, since they can be verbal as well as nominal. Once detected, the same mention-pair and mention-ranking models used for entities are often applied to events.

discourse deixis

An even more complex kind of coreference is **discourse deixis** (Webber, 1988), in which an anaphor refers back to a discourse segment, which can be quite hard to delimit or categorize, like the examples in (24.5) adapted from Webber (1991):

- (24.5) According to Soleil, Beau just opened a restaurant
- a. But *that* turned out to be a lie.
 - b. But *that* was false.
 - c. *That* struck me as a funny way to describe the situation.

The referent of *that* is a speech act (see Chapter 26) in (24.5a), a proposition in (24.5b), and a manner of description in (24.5c). We don't give algorithms in this chapter for these difficult types of **non-nominal antecedents**, but see Kolhatkar et al. (2018) for a survey.

24.1 Coreference Phenomena: Linguistic Background

We now offer some linguistic background on reference phenomena. We introduce the four types of referring expressions (definite and indefinite NPs, pronouns, and names), describe how these are used to evoke and access entities in the discourse model, and talk about linguistic features of the anaphor/antecedent relation (like number/gender agreement, or properties of verb semantics).

24.1.1 Types of Referring Expressions

Indefinite Noun Phrases: The most common form of indefinite reference in English is marked with the determiner *a* (or *an*), but it can also be marked by a quantifier such as *some* or even the determiner *this*. Indefinite reference generally introduces into the discourse context entities that are new to the hearer.

- (24.6) a. Mrs. Martin was so very kind as to send Mrs. Goddard *a beautiful goose*.
 b. He had gone round one day to bring her *some walnuts*.
 c. I saw *this beautiful cauliflower* today.

Definite Noun Phrases: Definite reference, such as via NPs that use the English article *the*, refers to an entity that is identifiable to the hearer. An entity can be identifiable to the hearer because it has been mentioned previously in the text and thus is already represented in the discourse model:

- (24.7) It concerns a white stallion which I have sold to an officer. But the pedigree of *the white stallion* was not fully established.

Alternatively, an entity can be identifiable because it is contained in the hearer's set of beliefs about the world, or the uniqueness of the object is implied by the description itself, in which case it evokes a representation of the referent into the discourse model, as in (24.9):

- (24.8) I read about it in the *New York Times*.
 (24.9) Have you seen the car keys?

These last uses are quite common; more than half of definite NPs in newswire texts are non-anaphoric, often because they are the first time an entity is mentioned (Poesio and Vieira 1998, Bean and Riloff 1999).

Pronouns: Another form of definite reference is pronominalization, used for entities that are extremely salient in the discourse, (as we discuss below):

- (24.10) Emma smiled and chatted as cheerfully as *she* could,

cataphora

Pronouns can also participate in **cataphora**, in which they are mentioned before their referents are, as in (24.11).

- (24.11) Even before *she* saw *it*, Dorothy had been thinking about the Emerald City every day.

Here, the pronouns *she* and *it* both occur *before* their referents are introduced.

bound

Pronouns also appear in quantified contexts in which they are considered to be **bound**, as in (24.12).

- (24.12) Every dancer brought *her* left arm forward.

Under the relevant reading, *her* does not refer to some woman in context, but instead behaves like a variable bound to the quantified expression *every dancer*. We are not concerned with the bound interpretation of pronouns in this chapter.

In some languages, pronouns can appear as clitics attached to a word, like *lo* (‘it’) in this Spanish example from AnCora (Recasens and Martí, 2010):

- (24.13) La intención es reconocer el gran prestigio que tiene la maratón y unirlo con esta gran carrera.
 ‘The aim is to recognize the great prestige that the Marathon has and join it with this great race.’

Demonstrative Pronouns: Demonstrative pronouns *this* and *that* can appear either alone or as determiners, for instance, *this ingredient*, *that spice*:

- (24.14) I just bought a copy of Thoreau’s *Walden*. I had bought one five years ago. *That one* had been very tattered; *this one* was in much better condition.

Note that *this NP* is ambiguous; in colloquial spoken English, it can be indefinite, as in (24.6), or definite, as in (24.14).

Zero Anaphora: Instead of using a pronoun, in some languages (including Chinese, Japanese, and Italian) it is possible to have an anaphor that has no lexical realization at all, called a **zero anaphor** or zero pronoun, as in the following Italian and Japanese examples from Poesio et al. (2016):

- (24.15) EN [John]_i went to visit some friends. On the way [he]_i bought some wine.
 IT [Giovanni]_i andò a far visita a degli amici. Per via ϕ_i comprò del vino.
 JA [John]_i-wa yujin-o houmon-sita. Tochu-de ϕ_i wain-o ka-tta.

or this Chinese example:

- (24.16) [我] 前一会精神上太紧张。[0] 现在比较平静了
 [I] was too nervous a while ago. ... [0] am now calmer.

Zero anaphors complicate the task of mention detection in these languages.

Names: Names (such as of people, locations, or organizations) can be used to refer to both new and old entities in the discourse:

- (24.17) a. **Miss Woodhouse** certainly had not done him justice.
 b. **International Business Machines** sought patent compensation from Amazon; **IBM** had previously sued other companies.

24.1.2 Information Status

information
status
discourse-new
discourse-old

The way referring expressions are used to evoke new referents into the discourse (introducing new information), or access old entities from the model (old information), is called their **information status** or **information structure**. Entities can be **discourse-new** or **discourse-old**, and indeed it is common to distinguish at least three kinds of entities informationally (Prince, 1981):

new NPs:

brand new NPs: these introduce entities that are discourse-new and hearer-new like *a fruit* or *some walnuts*.

unused NPs: these introduce entities that are discourse-new but hearer-old (like *Hong Kong*, *Marie Curie*, or *the New York Times*).

old NPs: also called **evoked NPs**, these introduce entities that already in the discourse model, hence are both discourse-old and hearer-old, like *it* in “*I went to a new restaurant. It was...*”.

inferables: these introduce entities that are neither hearer-old nor discourse-old, but the hearer can infer their existence by reasoning based on other entities that are in the discourse. Consider the following examples:

(24.18) I went to a superb restaurant yesterday. *The chef* had just opened it.

(24.19) Mix flour, butter and water. Knead *the dough* until shiny.

bridging
inference

Neither *the chef* nor *the dough* were in the discourse model based on the first sentence of either example, but the reader can make a **bridging inference** that these entities should be added to the discourse model and associated with the restaurant and the ingredients, based on world knowledge that restaurants have chefs and dough is the result of mixing flour and liquid (Haviland and Clark 1974, Webber and Baldwin 1992, Nissim et al. 2004, Hou et al. 2018).

given-new

The form of an NP gives strong clues to its information status. We often talk about an entity's position on the **given-new** dimension, the extent to which the referent is **given** (salient in the discourse, easier for the hearer to call to mind, predictable by the hearer), versus **new** (non-salient in the discourse, unpredictable) (Chafe 1976, Prince 1981, Gundel et al. 1993). A referent that is very **accessible** (Ariel, 2001) i.e., very salient in the hearer's mind or easy to call to mind, can be referred to with less linguistic material. For example pronouns are used only when the referent has a high degree of activation or **salience** in the discourse model.⁴ By contrast, less salient entities, like a new referent being introduced to the discourse, will need to be introduced with a longer and more explicit referring expression to help the hearer recover the referent.

accessible

salience

Thus when an entity is first introduced into a discourse its mentions are likely to have full names, titles or roles, or appositive or restrictive relative clauses, as in the introduction of our protagonist in (24.1): *Victoria Chen, CFO of Megabucks Banking*. As an entity is discussed over a discourse, it becomes more salient to the hearer and its mentions on average typically becomes shorter and less informative, for example with a shortened name (for example *Ms. Chen*), a definite description (*the 38-year-old*), or a pronoun (*she* or *her*) (Hawkins 1978). However, this change in length is not monotonic, and is sensitive to discourse structure (Grosz 1977b, Reichman 1985, Fox 1993).

24.1.3 Complications: Non-Referring Expressions

Many noun phrases or other nominals are not referring expressions, although they may bear a confusing superficial resemblance. For example in some of the earliest computational work on reference resolution, Karttunen (1969) pointed out that the NP *a car* in the following example does not create a discourse referent:

(24.20) Janet doesn't have *a car*.

and cannot be referred back to by anaphoric *it* or *the car*:

(24.21) **It* is a Toyota.

(24.22) **The car* is red.

We summarize here four common types of structures that are not counted as mentions in coreference tasks and hence complicate the task of mention-detection:

⁴ Pronouns also usually (but not always) refer to entities that were introduced no further than one or two sentences back in the ongoing discourse, whereas definite noun phrases can often refer further back.

Appositives: An appositional structure is a noun phrase that appears next to a head noun phrase, describing the head. In English they often appear in commas, like “a unit of UAL” appearing in apposition to the NP *United*, or *CFO of Megabucks Banking* in apposition to *Victoria Chen*.

(24.23) Victoria Chen, CFO of Megabucks Banking, saw ...

(24.24) United, a unit of UAL, matched the fares.

Appositional NPs are not referring expressions, instead functioning as a kind of supplementary parenthetical description of the head NP. Nonetheless, sometimes it is useful to link these phrases to an entity they describe, and so some datasets like OntoNotes mark appositional relationships.

Predicative and Prenominal NPs: Predicative or attributive NPs describe properties of the head noun. In *United is a unit of UAL*, the NP *a unit of UAL* describes a property of United, rather than referring to a distinct entity. Thus they are not marked as mentions in coreference tasks; in our example the NPs *\$2.3 million* and *the company’s president*, are attributive, describing properties of *her pay* and *the 38-year-old*; Example (24.27) shows a Chinese example in which the predicate NP (中国最大的城市; *China’s biggest city*) is not a mention.

(24.25) her pay jumped to *\$2.3 million*

(24.26) the 38-year-old became *the company’s president*

(24.27) 上海是[中国最大的城市] [Shanghai is *China’s biggest city*]

Expletives: Many uses of pronouns like *it* in English and corresponding pronouns in other languages are not referential. Such **expletive** or **pleonastic** cases include *it is raining*, in idioms like *hit it off*, or in particular syntactic situations like **clefts** (24.28a) or **extraposition** (24.28b):

(24.28) a. *It* was Emma Goldman who founded *Mother Earth*

b. *It* surprised me that there was a herring hanging on her wall.

Generics: Another kind of expression that does not refer back to an entity explicitly evoked in the text is *generic* reference. Consider (24.29).

(24.29) I love mangos. *They* are very tasty.

Here, *they* refers, not to a particular mango or set of mangos, but instead to the class of mangos in general. The pronoun *you* can also be used generically:

(24.30) In July in San Francisco *you* have to wear a jacket.

24.1.4 Linguistic Properties of the Coreference Relation

Now that we have seen the linguistic properties of individual referring expressions we turn to properties of the antecedent/anaphor pair. Understanding these properties is helpful both in designing novel features and performing error analyses.

Number Agreement: Referring expressions and their referents must generally agree in number; English *she/her/he/him/his/it* are singular, *we/us/they/them* are plural, and *you* is unspecified for number. So a plural antecedent like *the chefs* cannot generally corefer with a singular anaphor like *she*. However, algorithms cannot enforce number agreement too strictly. First, semantically plural entities can be referred to by either *it* or *they*:

(24.31) IBM announced a new machine translation product yesterday. *They* have been working on it for 20 years.

singular they

Second, **singular they** has become much more common, in which *they* is used to describe singular individuals, often useful because *they* is gender neutral. Although recently increasing, singular they is quite old, part of English for many centuries.⁵

Person Agreement: English distinguishes between first, second, and third person, and a pronoun's antecedent must agree with the pronoun in person. Thus a third person pronoun (*he, she, they, him, her, them, his, her, their*) must have a third person antecedent (one of the above or any other noun phrase). However, phenomena like quotation can cause exceptions; in this example *I, my*, and *she* are coreferent:

(24.32) "I voted for Nader because he was most aligned with my values," she said.

Gender or Noun Class Agreement: In many languages, all nouns have grammatical gender or noun class⁶ and pronouns generally agree with the grammatical gender of their antecedent. In English this occurs only with third-person singular pronouns, which distinguish between *male* (*he, him, his*), *female* (*she, her*), and *nonpersonal* (*it*) grammatical genders. Non-binary pronouns like *ze* or *hir* may also occur in more recent texts. Knowing which gender to associate with a name in text can be complex, and may require world knowledge about the individual. Some examples:

(24.33) Maryam has a theorem. She is exciting. (she=Maryam, not the theorem)

(24.34) Maryam has a theorem. It is exciting. (it=the theorem, not Maryam)

reflexive

Binding Theory Constraints: The **binding theory** is a name for syntactic constraints on the relations between a mention and an antecedent in the same sentence (Chomsky, 1981). Oversimplifying a bit, **reflexive** pronouns like *himself* and *herself* corefer with the subject of the most immediate clause that contains them (24.35), whereas nonreflexives cannot corefer with this subject (24.36).

(24.35) Janet bought herself a bottle of fish sauce. [herself=Janet]

(24.36) Janet bought her a bottle of fish sauce. [her≠Janet]

Recency: Entities introduced in recent utterances tend to be more salient than those introduced from utterances further back. Thus, in (24.37), the pronoun *it* is more likely to refer to Jim's map than the doctor's map.

(24.37) The doctor found an old map in the captain's chest. Jim found an even older map hidden on the shelf. It described an island.

Grammatical Role: Entities mentioned in subject position are more salient than those in object position, which are in turn more salient than those mentioned in oblique positions. Thus although the first sentence in (24.38) and (24.39) expresses roughly the same propositional content, the preferred referent for the pronoun *he* varies with the subject—Billy Bones in (24.38) and Jim Hawkins in (24.39).

(24.38) Billy Bones went to the bar with Jim Hawkins. He called for a glass of rum. [he = Billy]

(24.39) Jim Hawkins went to the bar with Billy Bones. He called for a glass of rum. [he = Jim]

⁵ Here's a bound pronoun example from Shakespeare's *Comedy of Errors*: *There's not a man I meet but doth salute me As if I were their well-acquainted friend*

⁶ The word "gender" is generally only used for languages with 2 or 3 noun classes, like most Indo-European languages; many languages, like the Bantu languages or Chinese, have a much larger number of noun classes.

Verb Semantics: Some verbs semantically emphasize one of their arguments, biasing the interpretation of subsequent pronouns. Compare (24.40) and (24.41).

(24.40) John telephoned Bill. He lost the laptop.

(24.41) John criticized Bill. He lost the laptop.

These examples differ only in the verb used in the first sentence, yet “he” in (24.40) is typically resolved to John, whereas “he” in (24.41) is resolved to Bill. This may be partly due to the link between implicit causality and saliency: the implicit cause of a “criticizing” event is its object, whereas the implicit cause of a “telephoning” event is its subject. In such verbs, the entity which is the implicit cause may be more salient.

Selectional Restrictions: Many other kinds of semantic knowledge can play a role in referent preference. For example, the selectional restrictions that a verb places on its arguments (Chapter 22) can help eliminate referents, as in (24.42).

(24.42) I ate the soup in my new bowl after cooking it for hours

There are two possible referents for *it*, the soup and the bowl. The verb *eat*, however, requires that its direct object denote something edible, and this constraint can rule out *bowl* as a possible referent.

24.2 Coreference Tasks and Datasets

We can formulate the task of coreference resolution as follows: Given a text T , find all entities and the coreference links between them. We evaluate our task by comparing the links our system creates with those in human-created gold coreference annotations on T .

Let’s return to our coreference example, now using superscript numbers for each coreference chain (cluster), and subscript letters for individual mentions in the cluster:

(24.43) [Victoria Chen]_a¹, CFO of [Megabucks Banking]_a², saw [[her]_b¹ pay]_a³ jump to \$2.3 million, as [the 38-year-old]_c¹ also became [[the company]_b²’s president. It is widely known that [she]_d¹ came to [Megabucks]_c² from rival [Lotsabucks]_a⁴.

Assuming example (24.43) was the entirety of the article, the chains for *her pay* and *Lotsabucks* are singleton mentions:

1. {Victoria Chen, her, the 38-year-old, She}
2. {Megabucks Banking, the company, Megabucks}
3. {her pay}
4. {Lotsabucks}

For most coreference evaluation campaigns, the input to the system is the raw text of articles, and systems must detect mentions and then link them into clusters. Solving this task requires dealing with pronominal anaphora (figuring out that *her* refers to *Victoria Chen*), filtering out non-referential pronouns like the pleonastic *It* in *It has been ten years*), dealing with definite noun phrases to figure out that *the 38-year-old* is coreferent with *Victoria Chen*, and that *the company* is the same as *Megabucks*. And we need to deal with names, to realize that *Megabucks* is the same as *Megabucks Banking*.

Exactly what counts as a mention and what links are annotated differs from task to task and dataset to dataset. For example some coreference datasets do not label singletons, making the task much simpler. Resolvers can achieve much higher scores on corpora without singletons, since singletons constitute the majority of mentions in running text, and they are often hard to distinguish from non-referential NPs. Some tasks use gold mention-detection (i.e. the system is given human-labeled mention boundaries and the task is just to cluster these gold mentions), which eliminates the need to detect and segment mentions from running text.

Coreference is usually evaluated by the **CoNLL F1** score, which combines three metrics: MUC, B^3 , and $CEAF_c$; Section 24.8 gives the details.

Let's mention a few characteristics of one popular coreference dataset, OntoNotes (Pradhan et al. 2007c, Pradhan et al. 2007a), and the CoNLL 2012 Shared Task based on it (Pradhan et al., 2012a). OntoNotes contains hand-annotated Chinese and English coreference datasets of roughly one million words each, consisting of newswire, magazine articles, broadcast news, broadcast conversations, web data and conversational speech data, as well as about 300,000 words of annotated Arabic newswire. The most important distinguishing characteristic of OntoNotes is that it does not label singletons, simplifying the coreference task, since singletons represent 60%-70% of all entities. In other ways, it is similar to other coreference datasets. Referring expression NPs that are coreferent are marked as mentions, but generics and pleonastic pronouns are not marked. Appositive clauses are not marked as separate mentions, but they are included in the mention. Thus in the NP, "Richard Godown, president of the Industrial Biotechnology Association" the mention is the entire phrase. Prenominal modifiers are annotated as separate entities only if they are proper nouns. Thus *wheat* is not an entity in *wheat fields*, but *UN* is an entity in *UN policy* (but not adjectives like *American* in *American policy*).

A number of corpora mark richer discourse phenomena. The ISNotes corpus annotates a portion of OntoNotes for information status, include bridging examples (Hou et al., 2018). The LitBank coreference corpus (Bamman et al., 2020) contains coreference annotations for 210,532 tokens from 100 different literary novels, including singletons and quantified and negated noun phrases. The AnCora-CO coreference corpus (Recasens and Martí, 2010) contains 400,000 words each of Spanish (AnCora-CO-Es) and Catalan (AnCora-CO-Ca) news data, and includes labels for complex phenomena like discourse deixis in both languages. The ARRAU corpus (Uryupina et al., 2020) contains 350,000 words of English marking all NPs, which means singleton clusters are available. ARRAU includes diverse genres like dialog (the TRAINS data) and fiction (the Pear Stories), and has labels for bridging references, discourse deixis, generics, and ambiguous anaphoric relations.

24.3 Mention Detection

mention detection

The first stage of coreference is **mention detection**: finding the spans of text that constitute each mention. Mention detection algorithms are usually very liberal in proposing candidate mentions (i.e., emphasizing recall), and only filtering later. For example many systems run parsers and named entity taggers on the text and extract every span that is either an **NP**, a **possessive pronoun**, or a **named entity**.

Doing so from our sample text repeated in (24.44):

(24.44) Victoria Chen, CFO of Megabucks Banking, saw her pay jump to \$2.3

million, as the 38-year-old also became the company's president. It is widely known that she came to Megabucks from rival Lotsabucks.

might result in the following list of 13 potential mentions:

Victoria Chen	\$2.3 million	she
CFO of Megabucks Banking	the 38-year-old	Megabucks
Megabucks Banking	the company	Lotsabucks
her	the company's president	
her pay	It	

More recent mention detection systems are even more generous; the span-based algorithm we will describe in Section 24.6 first extracts literally all n -gram spans of words up to $N=10$. Of course recall from Section 24.1.3 that many NPs—and the overwhelming majority of random n -gram spans—are not referring expressions. Therefore all such mention detection systems need to eventually filter out pleonastic/expletive pronouns like *It* above, appositives like *CFO of Megabucks Banking Inc.*, or predicate nominals like *the company's president* or *\$2.3 million*.

Some of this filtering can be done by rules. Early rule-based systems designed regular expressions to deal with pleonastic *it*, like the following rules from Lappin and Leass (1994) that use dictionaries of cognitive verbs (e.g., *believe*, *know*, *anticipate*) to capture pleonastic *it* in “It is *thought* that ketchup...”, or modal adjectives (e.g., *necessary*, *possible*, *certain*, *important*), for, e.g., “It is *likely* that I...”. Such rules are sometimes used as part of modern systems:

```
It is Modaladjective that S
It is Modaladjective (for NP) to VP
It is Cogv-ed that S
It seems/appears/means/follows (that) S
```

Mention-detection rules are sometimes designed specifically for particular evaluation campaigns. For OntoNotes, for example, mentions are not embedded within larger mentions, and while numeric quantities are annotated, they are rarely coreferential. Thus for OntoNotes tasks like CoNLL 2012 (Pradhan et al., 2012a), a common first pass rule-based mention detection algorithm (Lee et al., 2013) is:

1. Take all NPs, possessive pronouns, and named entities.
2. Remove numeric quantities (100 dollars, 8%), mentions embedded in larger mentions, adjectival forms of nations, and stop words (like *there*).
3. Remove pleonastic *it* based on regular expression patterns.

Rule-based systems, however, are generally insufficient to deal with mention-detection, and so modern systems incorporate some sort of learned mention detection component, such as a **referentiality** classifier, an **anaphoricity classifier**—detecting whether an NP is an anaphor—or a **discourse-new** classifier—detecting whether a mention is discourse-new and a potential antecedent for a future anaphor.

anaphoricity
detector

An **anaphoricity detector**, for example, can draw its positive training examples from any span that is labeled as an anaphoric referring expression in hand-labeled datasets like OntoNotes, ARRAU, or AnCora. Any other NP or named entity can be marked as a negative training example. Anaphoricity classifiers use features of the candidate mention such as its head word, surrounding words, definiteness, animacy, length, position in the sentence/discourse, many of which were first proposed in early work by Ng and Cardie (2002a); see Section 24.5 for more on features.

Referentiality or anaphoricity detectors can be run as filters, in which only mentions that are classified as anaphoric or referential are passed on to the coreference system. The end result of such a filtering mention detection system on our example above might be the following filtered set of 9 potential mentions:

Victoria Chen	her pay	she
Megabucks Bank	the 38-year-old	Megabucks
her	the company	Lotsabucks

It turns out, however, that hard filtering of mentions based on an anaphoricity or referentiality classifier leads to poor performance. If the anaphoricity classifier threshold is set too high, too many mentions are filtered out and recall suffers. If the classifier threshold is set too low, too many pleonastic or non-referential mentions are included and precision suffers.

The modern approach is instead to perform mention detection, anaphoricity, and coreference jointly in a single end-to-end model (Ng 2005b, Denis and Baldridge 2007, Rahman and Ng 2009). For example mention detection in the Lee et al. (2017b), 2018 system is based on a single end-to-end neural network that computes a score for each mention being referential, a score for two mentions being coreference, and combines them to make a decision, training all these scores with a single end-to-end loss. We'll describe this method in detail in Section 24.6.⁷

Despite these advances, correctly detecting referential mentions seems to still be an unsolved problem, since systems incorrectly marking pleonastic pronouns like *it* and other non-referential NPs as coreferent is a large source of errors of modern coreference resolution systems (Kummerfeld and Klein 2013, Martschat and Strube 2014, Martschat and Strube 2015, Wiseman et al. 2015, Lee et al. 2017a).

Mention, referentiality, or anaphoricity detection is thus an important open area of investigation. Other sources of knowledge may turn out to be helpful, especially in combination with unsupervised and semisupervised algorithms, which also mitigate the expense of labeled datasets. In early work, for example Bean and Riloff (1999) learned patterns for characterizing anaphoric or non-anaphoric NPs; (by extracting and generalizing over the first NPs in a text, which are guaranteed to be non-anaphoric). Chang et al. (2012) look for head nouns that appear frequently in the training data but never appear as gold mentions to help find non-referential NPs. Bergsma et al. (2008b) use web counts as a semisupervised way to augment standard features for anaphoricity detection for English *it*, an important task because *it* is both common and ambiguous; between a quarter and half *it* examples are non-anaphoric. Consider the following two examples:

(24.45) You can make [it] in advance. [anaphoric]

(24.46) You can make [it] in Hollywood. [non-anaphoric]

The *it* in *make it* is non-anaphoric, part of the idiom *make it*. Bergsma et al. (2008b) turn the context around each example into patterns, like “make * in advance” from (24.45), and “make * in Hollywood” from (24.46). They then use Google n-grams to enumerate all the words that can replace *it* in the patterns. Non-anaphoric contexts tend to only have *it* in the wildcard positions, while anaphoric contexts occur with many other NPs (for example *make them in advance* is just as frequent in their data

⁷ Some systems try to avoid mention detection or anaphoricity detection altogether. For datasets like OntoNotes which don't label singletons, an alternative to filtering out non-referential mentions is to run coreference resolution, and then simply delete any candidate mentions which were not corefered with another mention. This likely doesn't work as well as explicitly modeling referentiality, and cannot solve the problem of detecting singletons, which is important for tasks like entity linking.

as *make it in advance*, but *make them in Hollywood* did not occur at all). These n -gram contexts can be used as features in a supervised anaphoricity classifier.

24.4 Architectures for Coreference Algorithms

Modern systems for coreference are based on supervised neural machine learning, supervised from hand-labeled datasets like OntoNotes. In this section we overview the various architecture of modern systems, using the categorization of Ng (2010), which distinguishes algorithms based on whether they make each coreference decision in a way that is *entity-based*—representing each entity in the discourse model—or only *mention-based*—considering each mention independently, and whether they use *ranking models* to directly compare potential antecedents. Afterwards, we go into more detail on one state-of-the-art algorithm in Section 24.6.

24.4.1 The Mention-Pair Architecture

mention-pair

mention-pair

We begin with the **mention-pair** architecture, the simplest and most influential coreference architecture, which introduces many of the features of more complex algorithms, even though other architectures perform better. The **mention-pair** architecture is based around a classifier that—as its name suggests—is given a pair of mentions, a candidate anaphor and a candidate antecedent, and makes a binary classification decision: coreferring or not.

Let’s consider the task of this classifier for the pronoun *she* in our example, and assume the slightly simplified set of potential antecedents in Fig. 24.2.

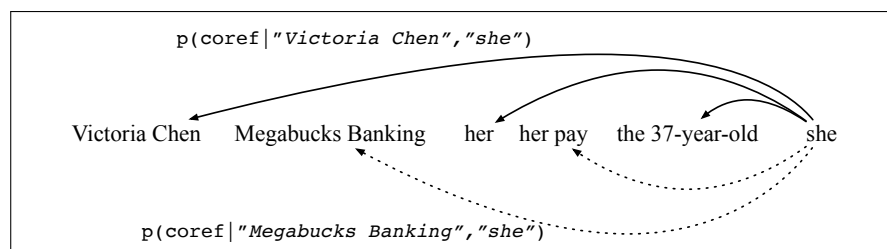


Figure 24.2 For each pair of a mention (like *she*), and a potential antecedent mention (like *Victoria Chen* or *her*), the mention-pair classifier assigns a probability of a coreference link.

For each prior mention (*Victoria Chen*, *Megabucks Banking*, *her*, etc.), the binary classifier computes a probability: whether or not the mention is the antecedent of *she*. We want this probability to be high for actual antecedents (*Victoria Chen*, *her*, *the 38-year-old*) and low for non-antecedents (*Megabucks Banking*, *her pay*).

Early classifiers used hand-built features (Section 24.5); more recent classifiers use neural representation learning (Section 24.6)

For training, we need a heuristic for selecting training samples; since most pairs of mentions in a document are not coreferent, selecting every pair would lead to a massive overabundance of negative samples. The most common heuristic, from (Soon et al., 2001), is to choose the closest antecedent as a positive example, and all pairs in between as the negative examples. More formally, for each anaphor mention m_i we create

- one positive instance (m_i, m_j) where m_j is the closest antecedent to m_i , and

- a negative instance (m_i, m_k) for each m_k between m_j and m_i

Thus for the anaphor *she*, we would choose (she, her) as the positive example and no negative examples. Similarly, for the anaphor *the company* we would choose $(the\ company, Megabucks)$ as the positive example and $(the\ company, she)$ $(the\ company, the\ 38\text{-year-old})$ $(the\ company, her\ pay)$ and $(the\ company, her)$ as negative examples.

Once the classifier is trained, it is applied to each test sentence in a clustering step. For each mention i in a document, the classifier considers each of the prior $i - 1$ mentions. In **closest-first** clustering (Soon et al., 2001), the classifier is run right to left (from mention $i - 1$ down to mention 1) and the first antecedent with probability $> .5$ is linked to i . If no antecedent has probably > 0.5 , no antecedent is selected for i . In **best-first** clustering, the classifier is run on all $i - 1$ antecedents and the most probable preceding mention is chosen as the antecedent for i . The transitive closure of the pairwise relation is taken as the cluster.

While the mention-pair model has the advantage of simplicity, it has two main problems. First, the classifier doesn't directly compare candidate antecedents to each other, so it's not trained to decide, between two likely antecedents, which one is in fact better. Second, it ignores the discourse model, looking only at mentions, not entities. Each classifier decision is made completely locally to the pair, without being able to take into account other mentions of the same entity. The next two models each address one of these two flaws.

24.4.2 The Mention-Rank Architecture

The mention ranking model directly compares candidate antecedents to each other, choosing the highest-scoring antecedent for each anaphor.

In early formulations, for mention i , the classifier decides which of the $\{1, \dots, i - 1\}$ prior mentions is the antecedent (Denis and Baldridge, 2008). But suppose i is in fact not anaphoric, and none of the antecedents should be chosen? Such a model would need to run a separate anaphoricity classifier on i . Instead, it turns out to be better to jointly learn anaphoricity detection and coreference together with a single loss (Rahman and Ng, 2009).

So in modern mention-ranking systems, for the i th mention (anaphor), we have an associated random variable y_i ranging over the values $Y(i) = \{1, \dots, i - 1, \epsilon\}$. The value ϵ is a special dummy mention meaning that i does not have an antecedent (i.e., is either discourse-new and starts a new coref chain, or is non-anaphoric).

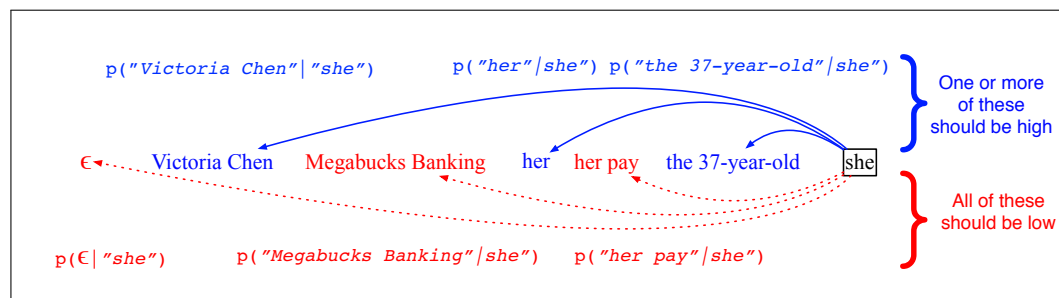


Figure 24.3 For each candidate anaphoric mention (like *she*), the mention-ranking system assigns a probability distribution over all previous mentions plus the special dummy mention ϵ .

At test time, for a given mention i the model computes one softmax over all the antecedents (plus ϵ) giving a probability for each candidate antecedent (or none).

Fig. 24.3 shows an example of the computation for the single candidate anaphor *she*.

Once the antecedent is classified for each anaphor, transitive closure can be run over the pairwise decisions to get a complete clustering.

Training is trickier in the mention-ranking model than the mention-pair model, because for each anaphor we don't know which of all the possible gold antecedents to use for training. Instead, the best antecedent for each mention is *latent*; that is, for each mention we have a whole cluster of legal gold antecedents to choose from. Early work used heuristics to choose an antecedent, for example choosing the closest antecedent as the gold antecedent and all non-antecedents in a window of two sentences as the negative examples (Denis and Baldridge, 2008). Various kinds of ways to model latent antecedents exist (Fernandes et al. 2012, Chang et al. 2013, Durrett and Klein 2013). The simplest way is to give credit to any legal antecedent by summing over all of them, with a loss function that optimizes the likelihood of all correct antecedents from the gold clustering (Lee et al., 2017b). We'll see the details in Section 24.6.

Mention-ranking models can be implemented with hand-built features or with neural representation learning (which might also incorporate some hand-built features). we'll explore both directions in Section 24.5 and Section 24.6.

24.4.3 Entity-based Models

Both the mention-pair and mention-ranking models make their decisions about *mentions*. By contrast, entity-based models link each mention not to a previous mention but to a previous discourse *entity* (cluster of mentions).

A mention-ranking model can be turned into an entity-ranking model simply by having the classifier make its decisions over clusters of mentions rather than individual mentions (Rahman and Ng, 2009).

For traditional feature-based models, this can be done by extracting features over clusters. The size of a cluster is a useful feature, as is its 'shape', which is the list of types of the mentions in the cluster i.e., sequences of the tokens (P)roper, (D)efinite, (I)ndefinite, (Pr)onoun, so that a cluster composed of {*Victoria, her, the 38-year-old*} would have the shape *P-Pr-D* (Björkelund and Kuhn, 2014). An entity-based model that includes a mention-pair classifier can use as features aggregates of mention-pair probabilities, for example computing the average probability of coreference over all mention-pairs in the two clusters (Clark and Manning 2015).

Neural models can learn representations of clusters automatically, for example by using an RNN over the sequence of cluster mentions to encode a state corresponding to a cluster representation (Wiseman et al., 2016), or by learning distributed representations for pairs of clusters by pooling over learned representations of mention pairs (Clark and Manning, 2016b).

However, although entity-based models are more expressive, the use of cluster-level information in practice has not led to large gains in performance, so mention-ranking models are still more commonly used.

24.5 Classifiers using hand-built features

Feature-based classifiers, use hand-designed features in logistic regression, SVM, or random forest classifiers for coreference resolution. These classifiers don't per-

form as well as neural ones. Nonetheless, they are still sometimes useful to build lightweight systems when compute or data are sparse, and the features themselves are useful for error analysis even in neural systems.

Given an anaphor mention and a potential antecedent mention, feature based classifiers make use of three types of features: (i) features of the anaphor, (ii) features of the candidate antecedent, and (iii) features of the relationship between the pair. Entity-based models can make additional use of two additional classes: (iv) feature of all mentions from the antecedent’s entity cluster, and (v) features of the relation between the anaphor and the mentions in the antecedent entity cluster.

Features of the Anaphor or Antecedent Mention		
First (last) word	Victoria/she	First or last word (or embedding) of antecedent/anaphor
Head word	Victoria/she	Head word (or head embedding) of antecedent/anaphor
Attributes	Sg-F-A-3-PER/ Sg-F-A-3-PER	The number, gender, animacy, person, named entity type attributes of (antecedent/anaphor)
Length	2/1	length in words of (antecedent/anaphor)
Mention type	P/Pr	Type: (P)roper, (D)efinite, (I)ndefinite, (Pr)onoun of antecedent/anaphor
Features of the Antecedent Entity		
Entity shape	P-Pr-D	The ‘shape’ or list of types of the mentions in the antecedent entity (cluster), i.e., sequences of (P)roper, (D)efinite, (I)ndefinite, (Pr)onoun.
Entity attributes	Sg-F-A-3-PER	The number, gender, animacy, person, named entity type attributes of the antecedent entity
Ant. cluster size	3	Number of mentions in the antecedent cluster
Features of the Pair of Mentions		
Sentence distance	1	The number of sentences between antecedent and anaphor
Mention distance	4	The number of mentions between antecedent and anaphor
i-within-i	F	Anaphor has i-within-i relation with antecedent
Cosine		Cosine between antecedent and anaphor embeddings
Features of the Pair of Entities		
Exact String Match	F	True if the strings of any two mentions from the antecedent and anaphor clusters are identical.
Head Word Match	F	True if any mentions from antecedent cluster has same headword as any mention in anaphor cluster
Word Inclusion	F	All words in anaphor cluster included in antecedent cluster

Figure 24.4 Feature-based coreference: sample feature values for anaphor “she” and potential antecedent “Victoria Chen”.

Figure 24.4 shows a selection of commonly used features, and shows the value that would be computed for the potential anaphor “she” and potential antecedent “Victoria Chen” in our example sentence, repeated below:

(24.47) **Victoria Chen**, CFO of Megabucks Banking, saw her pay jump to \$2.3 million, as the 38-year-old also became the company’s president. It is widely known that **she** came to Megabucks from rival Lotsabucks.

Features that prior work has found to be particularly useful are exact string match, entity headword agreement, mention distance, as well as (for pronouns) exact attribute match and i-within-i, and (for nominals and proper names) word inclusion and cosine. For lexical features (like head words) it is common to only use words that appear enough times (>20 times).

It is crucial in feature-based systems to use conjunctions of features; one experiment suggested that moving from individual features in a classifier to conjunctions of multiple features increased F1 by 4 points (Lee et al., 2017a). Specific conjunctions can be designed by hand (Durrett and Klein, 2013), all pairs of features can be conjoined (Bengtson and Roth, 2008), or feature conjunctions can be learned using decision tree or random forest classifiers (Ng and Cardie 2002a, Lee et al. 2017a).

Features can also be used in neural models as well. Neural systems use contextual word embeddings so don't benefit from shallow features like string match or mention types. However features like mention length, distance between mentions, or genre can complement neural contextual embedding models.

24.6 A neural mention-ranking algorithm

In this section we describe the neural **e2e-coref** algorithms of Lee et al. (2017b) (simplified and extended a bit, drawing on Joshi et al. (2019) and others). This is a **mention-ranking** algorithm that considers all possible spans of text in the document, assigns a mention-score to each span, prunes the mentions based on this score, then assigns coreference links to the remaining mentions.

More formally, given a document D with T words, the model considers all of the $\frac{T(T+1)}{2}$ text spans in D (unigrams, bigrams, trigrams, 4-grams, etc; in practice we only consider spans up a maximum length around 10). The task is to assign to each span i an antecedent y_i , a random variable ranging over the values $Y(i) = \{1, \dots, i-1, \epsilon\}$; each previous span and a special dummy token ϵ . Choosing the dummy token means that i does not have an antecedent, either because i is discourse-new and starts a new coreference chain, or because i is non-anaphoric.

For each pair of spans i and j , the system assigns a score $s(i, j)$ for the coreference link between span i and span j . The system then learns a distribution $P(y_i)$ over the antecedents for span i :

$$P(y_i) = \frac{\exp(s(i, y_i))}{\sum_{y' \in Y(i)} \exp(s(i, y'))} \quad (24.48)$$

This score $s(i, j)$ includes three factors that we'll define below: $m(i)$; whether span i is a mention; $m(j)$; whether span j is a mention; and $c(i, j)$; whether j is the antecedent of i :

$$s(i, j) = m(i) + m(j) + c(i, j) \quad (24.49)$$

For the dummy antecedent ϵ , the score $s(i, \epsilon)$ is fixed to 0. This way if any non-dummy scores are positive, the model predicts the highest-scoring antecedent, but if all the scores are negative it abstains.

24.6.1 Computing span representations

To compute the two functions $m(i)$ and $c(i, j)$ which score a span i or a pair of spans (i, j) , we'll need a way to represent a span. The e2e-coref family of algorithms represents each span by trying to capture 3 words/tokens: the first word, the last word, and the most important word. We first run each paragraph or subdocument through an encoder (like BERT) to generate embeddings h_i for each token i . The span i is then represented by a vector $\mathbf{g}_i$ that is a concatenation of the encoder output

embedding for the first (start) token of the span, the encoder output for the last (end) token of the span, and a third vector which is an attention-based representation:

$$\mathbf{g}_i = [\mathbf{h}_{\text{START}(i)}, \mathbf{h}_{\text{END}(i)}, \mathbf{h}_{\text{ATT}(i)}] \quad (24.50)$$

The goal of the attention vector is to represent which word/token is the likely syntactic head-word of the span; we saw in the prior section that head-words are a useful feature; a matching head-word is a good indicator of coreference. The attention representation is computed as usual; the system learns a weight vector $\mathbf{w}_\alpha$, and computes its dot product with the hidden state $\mathbf{h}_t$ transformed by a FFN:

$$\alpha_t = \mathbf{w}_\alpha \cdot \text{FFN}_\alpha(\mathbf{h}_t) \quad (24.51)$$

The attention score is normalized into a distribution via a softmax:

$$a_{i,t} = \frac{\exp(\alpha_t)}{\sum_{k=\text{START}(i)}^{\text{END}(i)} \exp(\alpha_k)} \quad (24.52)$$

And then the attention distribution is used to create a vector $\mathbf{h}_{\text{ATT}(i)}$ which is an attention-weighted sum of the embeddings $\mathbf{e}_t$ of each of the words in span i :

$$\mathbf{h}_{\text{ATT}(i)} = \sum_{t=\text{START}(i)}^{\text{END}(i)} a_{i,t} \cdot \mathbf{e}_t \quad (24.53)$$

Fig. 24.5 shows the computation of the span representation and the mention score.

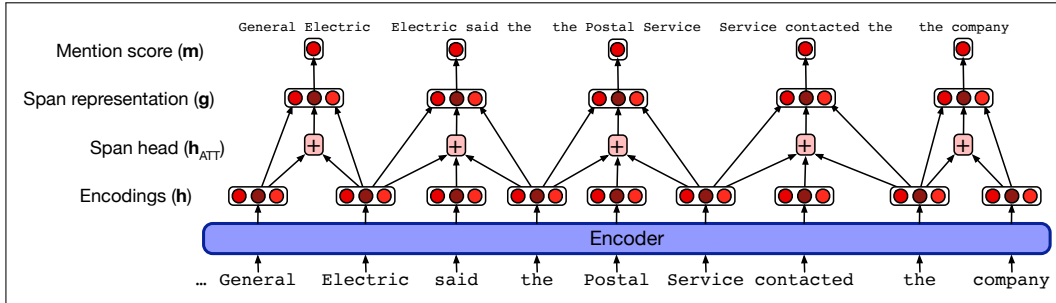


Figure 24.5 Computation of the span representation $\mathbf{g}$ (and the mention score $\mathbf{m}$) in a BERT version of the e2e-coref model (Lee et al. 2017b, Joshi et al. 2019). The model considers all spans up to a maximum width of say 10; the figure shows a small subset of the bigram and trigram spans.

24.6.2 Computing the mention and antecedent scores m and c

Now that we know how to compute the vector $\mathbf{g}_i$ for representing span i , we can see the details of the two scoring functions $m(i)$ and $c(i, j)$. Both are computed by feedforward networks:

$$m(i) = w_m \cdot \text{FFN}_m(\mathbf{g}_i) \quad (24.54)$$

$$c(i, j) = w_c \cdot \text{FFN}_c([\mathbf{g}_i, \mathbf{g}_j, \mathbf{g}_i \circ \mathbf{g}_j]) \quad (24.55)$$

At inference time, this mention score m is used as a filter to keep only the best few mentions.

We then compute the antecedent score for high-scoring mentions. The antecedent score $c(i, j)$ takes as input a representation of the spans i and j , but also the element-wise similarity of the two spans to each other $\mathbf{g}_i \circ \mathbf{g}_j$ (here $\circ$ is element-wise multiplication). Fig. 24.6 shows the computation of the score s for the three possible antecedents of *the company* in the example sentence from Fig. 24.5.

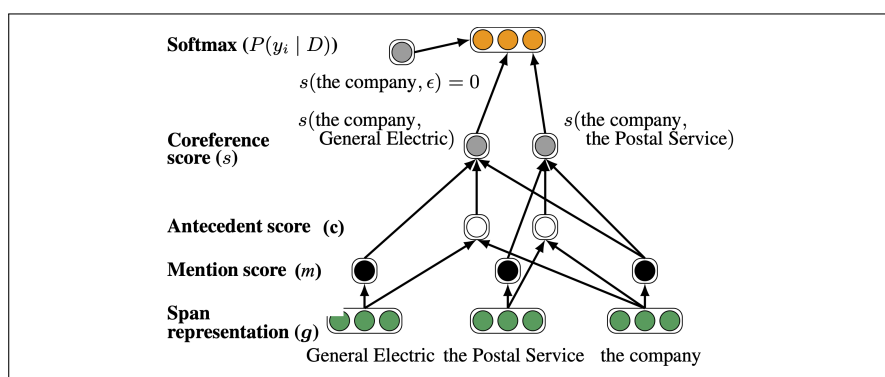


Figure 24.6 The computation of the score s for the three possible antecedents of *the company* in the example sentence from Fig. 24.5. Figure after Lee et al. (2017b).

Given the set of mentions, the joint distribution of antecedents for each document is computed in a forward pass, and we can then do transitive closure on the antecedents to create a final clustering for the document.

Fig. 24.7 shows example predictions from the model, showing the attention weights, which Lee et al. (2017b) find correlate with traditional semantic heads. Note that the model gets the second example wrong, presumably because *attendants* and *pilot* likely have nearby word embeddings.

We are looking for (a **region of central Italy bordering the Adriatic Sea**). (The **area**) is mostly mountainous and includes Mt. Corno, the highest peak of the Apennines. (It) also includes a lot of sheep, good clean-living, healthy sheep, and an Italian entrepreneur has an idea about how to make a little money of them.

(The **flight attendants**) have until 6:00 today to ratify labor concessions. (The **pilots'**) union and ground crew did so yesterday.

Figure 24.7 Sample predictions from the Lee et al. (2017b) model, with one cluster per example, showing one correct example and one mistake. Bold, parenthesized spans are mentions in the predicted cluster. The amount of red color on a word indicates the head-finding attention weight $a_{i,t}$ in Eq. 24.52. Figure adapted from Lee et al. (2017b).

24.6.3 Learning

For training, we don't have a single gold antecedent for each mention; instead the coreference labeling only gives us each entire cluster of coreferent mentions; so a mention only has a latent antecedent. We therefore use a loss function that maximizes the sum of the coreference probability of any of the legal antecedents. For a given mention i with possible antecedents $Y(i)$, let $\text{GOLD}(i)$ be the set of mentions in the gold cluster containing i . Since the set of mentions occurring before i is $Y(i)$, the set of mentions in that gold cluster that also occur before i is $Y(i) \cap \text{GOLD}(i)$. We

therefore want to maximize:

$$\sum_{\hat{y} \in Y(i) \cap \text{GOLD}(i)} P(\hat{y}) \quad (24.56)$$

If a mention i is not in a gold cluster $\text{GOLD}(i) = \epsilon$.

To turn this probability into a loss function, we'll use the cross-entropy loss function we defined in Eq. 4.19 in Chapter 4, by taking the $-\log$ of the probability. If we then sum over all mentions, we get the final loss function for training:

$$L = \sum_{i=2}^N -\log \sum_{\hat{y} \in Y(i) \cap \text{GOLD}(i)} P(\hat{y}) \quad (24.57)$$

24.7 Entity Linking

entity linking

Entity linking is the task of associating a mention in text with the representation of some real-world entity in an ontology or knowledge base (Ji and Grishman, 2011). It is the natural follow-on to coreference resolution; coreference resolution is the task of associating textual mentions that corefer to the same entity. Entity linking takes the further step of identifying who that entity is. It is especially important for any NLP task that links to a knowledge base.

While there are all sorts of potential knowledge-bases, we'll focus in this section on Wikipedia, since it's widely used as an ontology for NLP tasks. In this usage, each unique Wikipedia page acts as the unique id for a particular entity. This task of deciding which Wikipedia page corresponding to an individual is being referred to by a text mention has its own name: **wikification** (Mihalcea and Csomai, 2007).

wikification

Since the earliest systems (Mihalcea and Csomai 2007, Cucerzan 2007, Milne and Witten 2008), entity linking is done in (roughly) two stages: **mention detection** and **mention disambiguation**. We'll give two algorithms, one simple classic baseline that uses **anchor dictionaries** and information from the Wikipedia graph structure (Ferragina and Scaiella, 2011) and one modern neural algorithm (Li et al., 2020). We'll focus here mainly on the application of entity linking to questions, since a lot of the literature has been in that context.

24.7.1 Linking based on Anchor Dictionaries and Web Graph

As a simple baseline we introduce the TAGME linker (Ferragina and Scaiella, 2011) for Wikipedia, which itself draws on earlier algorithms (Mihalcea and Csomai 2007, Cucerzan 2007, Milne and Witten 2008). Wikification algorithms define the set of entities as the set of Wikipedia pages, so we'll refer to each Wikipedia page as a unique entity e . TAGME first creates a catalog of all entities (i.e. all Wikipedia pages, removing some disambiguation and other meta-pages) and indexes them in a standard IR engine like Lucene. For each page e , the algorithm computes an **in-link** count $\text{in}(e)$: the total number of in-links from other Wikipedia pages that point to e . These counts can be derived from Wikipedia dumps.

anchor texts

Finally, the algorithm requires an **anchor dictionary**. An anchor dictionary lists for each Wikipedia page, its **anchor texts**: the hyperlinked spans of text on other pages that point to it. For example, the web page for Stanford University, <http://www.stanford.edu>, might be pointed to from another page using anchor texts like *Stanford* or *Stanford University*:

Stanford University

We compute a Wikipedia anchor dictionary by including, for each Wikipedia page e , e 's title as well as all the anchor texts from all Wikipedia pages that point to e . For each anchor string a we'll also compute its total frequency $\text{freq}(a)$ in Wikipedia (including non-anchor uses), the number of times a occurs as a link (which we'll call $\text{link}(a)$), and its link probability $\text{linkprob}(a) = \text{link}(a) / \text{freq}(a)$. Some cleanup of the final anchor dictionary is required, for example removing anchor strings composed only of numbers or single characters, that are very rare, or that are very unlikely to be useful entities because they have a very low linkprob .

Mention Detection Given a question (or other text we are trying to link), TAGME detects mentions by querying the anchor dictionary for each token sequence up to 6 words. This large set of sequences is pruned with some simple heuristics (for example pruning substrings if they have small linkprobs). The question:

When was Ada Lovelace born?

might give rise to the anchor *Ada Lovelace* and possibly *Ada*, but substrings spans like *Lovelace* might be pruned as having too low a linkprob , and but spans like *born* have such a low linkprob that they would not be in the anchor dictionary at all.

Mention Disambiguation If a mention span is unambiguous (points to only one entity/Wikipedia page), we are done with entity linking! However, many spans are ambiguous, matching anchors for multiple Wikipedia entities/pages. The TAGME algorithm uses two factors for disambiguating ambiguous spans, which have been referred to as *prior probability* and *relatedness/coherence*. The first factor is $p(e|a)$, the probability with which the span refers to a particular entity. For each page $e \in \mathcal{E}(a)$, the probability $p(e|a)$ that anchor a points to e , is the ratio of the number of links into e with anchor text a to the total number of occurrences of a as an anchor:

$$\text{prior}(a \rightarrow e) = p(e|a) = \frac{\text{count}(a \rightarrow e)}{\text{link}(a)} \quad (24.58)$$

Let's see how that factor works in linking entities in the following question:

What Chinese Dynasty came before the Yuan?

The most common association for the span *Yuan* in the anchor dictionary is the name of the Chinese currency, i.e., the probability $p(\text{Yuan_currency} | \text{yuan})$ is very high. Rarer Wikipedia associations for *Yuan* include the common Chinese last name, a language spoken in Thailand, and the correct entity in this case, the name of the Chinese dynasty. So if we chose based only on $p(e|a)$, we would make the wrong disambiguation and miss the correct link, *Yuan_dynasty*.

To help in just this sort of case, TAGME uses a second factor, the **relatedness** of this entity to other entities in the input question. In our example, the fact that the question also contains the span *Chinese Dynasty*, which has a high probability link to the page *Dynasties_in_Chinese_history*, ought to help match *Yuan_dynasty*.

Let's see how this works. Given a question q , for each candidate anchors span a detected in q , we assign a relatedness score to each possible entity $e \in \mathcal{E}(a)$ of a . The relatedness score of the link $a \rightarrow e$ is the weighted average relatedness between e and all other entities in q . Two entities are considered related to the extent their Wikipedia pages share many in-links. More formally, the relatedness between two entities A and B is computed as

$$\text{rel}(A, B) = \frac{\log(\max(|\text{in}(A)|, |\text{in}(B)|)) - \log(|\text{in}(A) \cap \text{in}(B)|)}{\log(|W|) - \log(\min(|\text{in}(A)|, |\text{in}(B)|))} \quad (24.59)$$

where $\text{in}(x)$ is the set of Wikipedia pages pointing to x and W is the set of all Wikipedia pages in the collection.

The vote given by anchor b to the candidate annotation $a \rightarrow X$ is the average, over all the possible entities of b , of their relatedness to X , weighted by their prior probability:

$$\text{vote}(b, X) = \frac{1}{|\mathcal{E}(b)|} \sum_{Y \in \mathcal{E}(b)} \text{rel}(X, Y) p(Y|b) \quad (24.60)$$

The total relatedness score for $a \rightarrow X$ is the sum of the votes of all the other anchors detected in q :

$$\text{relatedness}(a \rightarrow X) = \sum_{b \in \mathcal{X}_q \setminus a} \text{vote}(b, X) \quad (24.61)$$

To score $a \rightarrow X$, we combine relatedness and prior by choosing the entity X that has the highest $\text{relatedness}(a \rightarrow X)$, finding other entities within a small ϵ of this value, and from this set, choosing the entity with the highest prior $P(X|a)$. The result of this step is a single entity assigned to each span in q .

The TAGME algorithm has one further step of pruning spurious anchor/entity pairs, assigning a score averaging link probability with the coherence.

$$\begin{aligned} \text{coherence}(a \rightarrow X) &= \frac{1}{|S| - 1} \sum_{B \in S \setminus X} \text{rel}(B, X) \\ \text{score}(a \rightarrow X) &= \frac{\text{coherence}(a \rightarrow X) + \text{linkprob}(a)}{2} \end{aligned} \quad (24.62)$$

Finally, pairs are pruned if $\text{score}(a \rightarrow X) < \lambda$, where the threshold λ is set on a held-out set.

24.7.2 Neural Graph-based linking

More recent entity linking models are based on **bi-encoders**, encoding a candidate mention span, encoding an entity, and computing the dot product between the encodings. This allows embeddings for all the entities in the knowledge base to be precomputed and cached (Wu et al., 2020). Let's sketch the ELQ linking algorithm of Li et al. (2020), which is given a question q and a set of candidate entities from Wikipedia with associated Wikipedia text, and outputs tuples (e, m_s, m_e) of entity id, mention start, and mention end. As Fig. 24.8 shows, it does this by encoding each Wikipedia entity using text from Wikipedia, encoding each mention span using text from the question, and computing their similarity, as we describe below.

Entity Mention Detection To get an h -dimensional embedding for each question token, the algorithm runs the question through BERT in the normal way:

$$[\mathbf{q}_1 \cdots \mathbf{q}_n] = \text{BERT}([\text{CLS}]q_1 \cdots q_n[\text{SEP}]) \quad (24.63)$$

It then computes the likelihood of each span $[i, j]$ in q being an entity mention, in a way similar to the span-based algorithm we saw for the reader above. First we compute the score for i/j being the start/end of a mention:

$$s_{\text{start}}(i) = \mathbf{w}_{\text{start}} \cdot \mathbf{q}_i, \quad s_{\text{end}}(j) = \mathbf{w}_{\text{end}} \cdot \mathbf{q}_j, \quad (24.64)$$

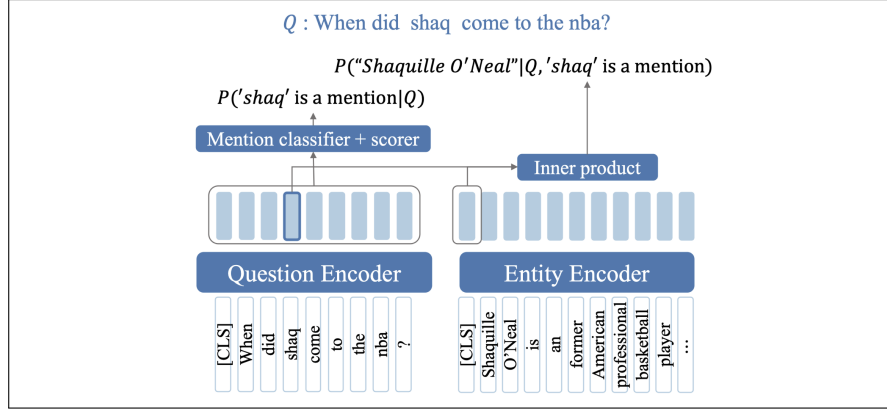


Figure 24.8 A sketch of the inference process in the ELQ algorithm for entity linking in questions (Li et al., 2020). Each candidate question mention span and candidate entity are separately encoded, and then scored by the entity/span dot product.

where $\mathbf{w}_{\text{start}}$ and $\mathbf{w}_{\text{end}}$ are vectors learned during training. Next, another trainable embedding, $\mathbf{w}_{\text{mention}}$ is used to compute a score for each token being part of a mention:

$$s_{\text{mention}}(t) = \mathbf{w}_{\text{mention}} \cdot \mathbf{q}_t \quad (24.65)$$

Mention probabilities are then computed by combining these three scores:

$$p([i, j]) = \sigma \left(s_{\text{start}}(i) + s_{\text{end}}(j) + \sum_{t=i}^j s_{\text{mention}}(t) \right) \quad (24.66)$$

Entity Linking To link mentions to entities, we next compute embeddings for each entity in the set $\mathcal{E} = e_1, \dots, e_i, \dots, e_w$ of all Wikipedia entities. For each entity e_i we'll get text from the entity's Wikipedia page, the title $t(e_i)$ and the first 128 tokens of the Wikipedia page which we'll call the description $d(e_i)$. This is again run through BERT, taking the output of the CLS token $\text{BERT}_{[\text{CLS}]}$ as the entity representation:

$$\mathbf{x}_{e_i} = \text{BERT}_{[\text{CLS}]}([\text{CLS}]t(e_i)[\text{ENT}]d(e_i)[\text{SEP}]) \quad (24.67)$$

Mention spans can be linked to entities by computing, for each entity e and span $[i, j]$, the dot product similarity between the span encoding (the average of the token embeddings) and the entity encoding.

$$\mathbf{y}_{i,j} = \frac{1}{(j-i+1)} \sum_{t=i}^j \mathbf{q}_t$$

$$s(e, [i, j]) = \mathbf{x}_e \cdot \mathbf{y}_{i,j} \quad (24.68)$$

Finally, we take a softmax to get a distribution over entities for each span:

$$p(e|[i, j]) = \frac{\exp(s(e, [i, j]))}{\sum_{e' \in \mathcal{E}} \exp(s(e', [i, j]))} \quad (24.69)$$

Training The ELQ mention detection and entity linking algorithm is fully supervised. This means, unlike the anchor dictionary algorithms from Section 24.7.1,

it requires datasets with entity boundaries marked and linked. Two such labeled datasets are WebQuestionsSP (Yih et al., 2016), an extension of the WebQuestions (Berant et al., 2013) dataset derived from Google search questions, and GraphQuestions (Su et al., 2016). Both have had entity spans in the questions marked and linked (Sorokin and Gurevych 2018, Li et al. 2020) resulting in entity-labeled versions WebQSP_{EL} and GraphQ_{EL} (Li et al., 2020).

Given a training set, the ELQ mention detection and entity linking phases are trained jointly, optimizing the sum of their losses. The mention detection loss is a binary cross-entropy loss, with L the length of the passage and N the number of candidates:

$$\mathcal{L}_{\text{MD}} = -\frac{1}{N} \sum_{1 \leq i \leq j \leq \min(i+L-1, n)} (y_{[i,j]} \log p([i,j]) + (1 - y_{[i,j]}) \log(1 - p([i,j]))) \quad (24.70)$$

with $y_{[i,j]} = 1$ if $[i, j]$ is a gold mention span, else 0. The entity linking loss is:

$$\mathcal{L}_{\text{ED}} = -\log p(e_g | [i, j]) \quad (24.71)$$

where e_g is the gold entity for mention $[i, j]$.

24.8 Evaluation of Coreference Resolution

We evaluate coreference algorithms model-theoretically, comparing a set of **hypothesis** chains or clusters H produced by the system against a set of gold or **reference** chains or clusters R from a human labeling, and reporting precision and recall.

However, there are a wide variety of methods for doing this comparison. In fact, there are 5 common metrics used to evaluate coreference algorithms: the **link** based MUC (Vilain et al., 1995) and BLANC (Recasens and Hovy 2011, Luo et al. 2014) metrics, the **mention** based B^3 metric (Bagga and Baldwin, 1998), the **entity** based CEAF metric (Luo, 2005), and the **link** based **entity** aware LEA metric (Moosavi and Strube, 2016).

MUC
F-measure

Let's just explore two of the metrics. The **MUC F-measure** (Vilain et al., 1995) is based on the number of coreference *links* (pairs of mentions) common to H and R . Precision is the number of common links divided by the number of links in H . Recall is the number of common links divided by the number of links in R ; This makes MUC biased toward systems that produce large chains (and fewer entities), and it ignores singletons, since they don't involve links.

B^3

B^3 is mention-based rather than link-based. For each mention in the reference chain, we compute a precision and recall, and then we take a weighted sum over all N mentions in the document to compute a precision and recall for the entire task. For a given mention i , let R be the reference chain that includes i , and H the hypothesis chain that has i . The set of correct mentions in H is $H \cap R$. Precision for mention i is thus $\frac{|H \cap R|}{|H|}$, and recall for mention i thus $\frac{|H \cap R|}{|R|}$. The total precision is the weighted sum of the precision for mention i , weighted by a weight w_i . The total recall is the weighted sum of the recall for mention i , weighted by a weight w_i . Equivalently:

$$\begin{aligned} \text{Precision} &= \sum_{i=1}^N w_i \frac{\# \text{ of correct mentions in hypothesis chain containing entity}_i}{\# \text{ of mentions in hypothesis chain containing entity}_i} \\ \text{Recall} &= \sum_{i=1}^N w_i \frac{\# \text{ of correct mentions in hypothesis chain containing entity}_i}{\# \text{ of mentions in reference chain containing entity}_i} \end{aligned}$$

The weight w_i for each entity can be set to different values to produce different versions of the algorithm.

Following a proposal from [Denis and Baldridge \(2009\)](#), the CoNLL coreference competitions were scored based on the average of MUC, CEAF-e, and B³ ([Pradhan et al. 2011](#), [Pradhan et al. 2012b](#)), and so it is common in many evaluation campaigns to report an average of these 3 metrics. See [Luo and Pradhan \(2016\)](#) for a detailed description of the entire set of metrics; reference implementations of these should be used rather than attempting to reimplement from scratch ([Pradhan et al., 2014](#)).

Alternative metrics have been proposed that deal with particular coreference domains or tasks. For example, consider the task of resolving mentions to named entities (persons, organizations, geopolitical entities), which might be useful for information extraction or knowledge base completion. A hypothesis chain that correctly contains all the pronouns referring to an entity, but has no version of the name itself, or is linked with a wrong name, is not useful for this task. We might instead want a metric that weights each mention by how informative it is (with names being most informative) ([Chen and Ng, 2013](#)) or a metric that considers a hypothesis to match a gold chain only if it contains at least one variant of a name (the NEC F1 metric of [Agarwal et al. \(2019\)](#)).

24.9 Winograd Schema problems

From early on in the field, researchers have noted that some cases of coreference are quite difficult, seeming to require world knowledge or sophisticated reasoning to solve. The problem was most famously pointed out by [Winograd \(1972\)](#) with the following example:

- (24.72) The city council denied the demonstrators a permit because
- a. they feared violence.
 - b. they advocated violence.

Winograd noticed that the antecedent that most readers preferred for the pronoun *they* in continuation (a) was *the city council*, but in (b) was *the demonstrators*. He suggested that this requires understanding that the second clause is intended as an explanation of the first clause, and also that our cultural frames suggest that city councils are perhaps more likely than demonstrators to fear violence and that demonstrators might be more likely to advocate violence.

Winograd
schema

In an attempt to get the field of NLP to focus more on methods involving world knowledge and common-sense reasoning, [Levesque \(2011\)](#) proposed a challenge task called the **Winograd Schema Challenge**.⁸ The problems in the challenge task are coreference problems designed to be easily disambiguated by the human reader, but hopefully not solvable by simple techniques such as selectional restrictions, or other basic word association methods.

The problems are framed as a pair of statements that differ in a single word or phrase, and a coreference question:

- (24.73) The trophy didn't fit into the suitcase because it was too **large**.

Question: What was too **large**? Answer: The trophy

⁸ Levesque's call was quickly followed up by [Levesque et al. \(2012\)](#) and [Rahman and Ng \(2012\)](#), a competition at the IJCAI conference ([Davis et al., 2017](#)), and a natural language inference version of the problem called WNLI ([Wang et al., 2018a](#)).

(24.74) The trophy didn't fit into the suitcase because it was too **small**.

Question: What was too **small**? Answer: The suitcase

The problems have the following characteristics:

1. The problems each have two parties
2. A pronoun preferentially refers to one of the parties, but could grammatically also refer to the other
3. A question asks which party the pronoun refers to
4. If one word in the question is changed, the human-preferred answer changes to the other party

The kind of world knowledge that might be needed to solve the problems can vary. In the trophy/suitcase example, it is knowledge about the physical world; that a bigger object cannot fit into a smaller object. In the original Winograd sentence, it is stereotypes about social actors like politicians and protesters. In examples like the following, it is knowledge about human actions like turn-taking or thanking.

(24.75) Bill passed the gameboy to John because his turn was [over/next]. Whose turn was [over/next]? Answers: Bill/John

(24.76) Joan made sure to thank Susan for all the help she had [given/received]. Who had [given/received] help? Answers: Susan/Joan.

Although the Winograd Schema was designed to require common-sense reasoning, a large percentage of the original set of problems can be solved by pre-trained language models, fine-tuned on Winograd Schema sentences (Kocijan et al., 2019). Large pretrained language models encode an enormous amount of world or common-sense knowledge! The current trend is therefore to propose new datasets with increasingly difficult Winograd-like coreference resolution problems like KNOWREF (Emami et al., 2019), with examples like:

(24.77) Marcus is undoubtedly faster than Jarrett right now but in [his] prime the gap wasn't all that big.

In the end, it seems likely that some combination of language modeling and knowledge will prove fruitful; indeed, it seems that knowledge-based models overfit less to lexical idiosyncracies in Winograd Schema training sets (Trichelair et al., 2018),

24.10 Gender Bias in Coreference

As with other aspects of language processing, coreference models exhibit gender and other biases (Zhao et al. 2018a, Rudinger et al. 2018, Webster et al. 2018). For example the WinoBias dataset (Zhao et al., 2018a) uses a variant of the Winograd Schema paradigm to test the extent to which coreference algorithms are biased toward linking gendered pronouns with antecedents consistent with cultural stereotypes. As we summarized in Chapter 5, embeddings replicate societal biases in their training test, such as associating men with historically stereotypical male occupations like doctors, and women with stereotypical female occupations like secretaries (Caliskan et al. 2017, Garg et al. 2018).

A WinoBias sentence contain two mentions corresponding to stereotypically-male and stereotypically-female occupations and a gendered pronoun that must be linked to one of them. The sentence cannot be disambiguated by the gender of the pronoun, but a biased model might be distracted by this cue. Here is an example sentence:

(24.78) The secretary called the physician_i and told him_i about a new patient
[pro-stereotypical]

(24.79) The secretary called the physician_i and told her_i about a new patient
[anti-stereotypical]

Zhao et al. (2018a) consider a coreference system to be biased if it is more accurate at linking pronouns consistent with gender stereotypical occupations (e.g., *him* with *physician* in (24.78)) than linking pronouns inconsistent with gender-stereotypical occupations (e.g., *her* with *physician* in (24.79)). They show that coreference systems of all architectures (rule-based, feature-based machine learned, and end-to-end-neural) all show significant bias, performing on average 21 F₁ points worse in the anti-stereotypical cases.

One possible source of this bias is that female entities are significantly underrepresented in the OntoNotes dataset, used to train most coreference systems. Zhao et al. (2018a) propose a way to overcome this bias: they generate a second gender-swapped dataset in which all male entities in OntoNotes are replaced with female ones and vice versa, and retrain coreference systems on the combined original and swapped OntoNotes data, also using debiased GloVe embeddings (Bolukbasi et al., 2016). The resulting coreference systems no longer exhibit bias on the WinoBias dataset, without significantly impacting OntoNotes coreference accuracy. In a follow-up paper, Zhao et al. (2019) show that the same biases exist in ELMo contextualized word vector representations and coref systems that use them. They showed that retraining ELMo with data augmentation again reduces or removes bias in coreference systems on WinoBias.

Webster et al. (2018) introduces another dataset, GAP, and the task of Gendered Pronoun Resolution as a tool for developing improved coreference algorithms for gendered pronouns. GAP is a gender-balanced labeled corpus of 4,454 sentences with gendered ambiguous pronouns (by contrast, only 20% of the gendered pronouns in the English OntoNotes training data are feminine). The examples were created by drawing on naturally occurring sentences from Wikipedia pages to create hard to resolve cases with two named entities of the same gender and an ambiguous pronoun that may refer to either person (or neither), like the following:

(24.80) In May, Fujisawa joined Mari Motohashi's rink as the team's skip, moving back from Karuizawa to Kitami where **she** had spent her junior days.

Webster et al. (2018) show that modern coreference algorithms perform significantly worse on resolving feminine pronouns than masculine pronouns in GAP. Kurita et al. (2019) shows that a system based on BERT contextualized word representations shows similar bias.

24.11 Summary

This chapter introduced the task of **coreference resolution**.

- This is the task of linking together **mentions** in text which **corefer**, i.e. refer to the same **discourse entity** in the **discourse model**, resulting in a set of coreference **chains** (also called **clusters** or **entities**).
- Mentions can be **definite NPs** or **indefinite NPs**, **pronouns** (including **zero pronouns**) or **names**.

- The surface form of an entity mention is linked to its **information status** (**new**, **old**, or **inferable**), and how **accessible** or **salient** the entity is.
- Some NPs are not referring expressions, such as pleonastic *it* in *It is raining*.
- Many corpora have human-labeled coreference annotations that can be used for supervised learning, including **OntoNotes** for English, Chinese, and Arabic, ARRAU for English, and **AnCora** for Spanish and Catalan.
- Mention detection can start with all nouns and named entities and then use **anaphoricity classifiers** or **referentiality classifiers** to filter out non-mentions.
- Three common architectures for coreference are **mention-pair**, **mention-rank**, and **entity-based**, each of which can make use of feature-based or neural classifiers.
- Modern coreference systems tend to be end-to-end, performing mention detection and coreference in a single end-to-end architecture.
- Algorithms learn representations for text spans and heads, and learn to compare anaphor spans with candidate antecedent spans.
- Entity linking is the task of associating a mention in text with the representation of some real-world entity in an ontology.
- Coreference systems are evaluated by comparing with gold entity labels using precision/recall metrics like **MUC**, **B³**, **CEAF**, **BLANC**, or **LEA**.
- The **Winograd Schema Challenge** problems are difficult coreference problems that seem to require world knowledge or sophisticated reasoning to solve.
- Coreference systems exhibit **gender bias** which can be evaluated using datasets like Winobias and GAP.

Historical Notes

Coreference has been part of natural language processing since the 1970s (Woods et al. 1972, Winograd 1972). The discourse model and the entity-centric foundation of coreference was formulated by Karttunen (1969) (at the 3rd COLING conference), playing a role also in linguistic semantics (Heim 1982, Kamp 1981). But it was Bonnie Webber’s 1978 dissertation and following work (Webber 1983) that explored the model’s computational aspects, providing fundamental insights into how entities are represented in the discourse model and the ways in which they can license subsequent reference. Many of the examples she provided continue to challenge theories of reference to this day.

Hobbs
algorithm

The **Hobbs algorithm**⁹ is a tree-search algorithm that was the first in a long series of syntax-based methods for identifying reference robustly in naturally occurring text. The input to the Hobbs algorithm is a pronoun to be resolved, together with a syntactic (constituency) parse of the sentences up to and including the current sentence. The details of the algorithm depend on the grammar used, but can be understood from a simplified version due to Kehler et al. (2004) that just searches through the list of NPs in the current and prior sentences. This simplified Hobbs algorithm searches NPs in the following order: “(i) in the current sentence from right-to-left, starting with the first NP to the left of the pronoun, (ii) in the previous sentence from left-to-right, (iii) in two sentences prior from left-to-right, and (iv) in

⁹ The simpler of two algorithms presented originally in Hobbs (1978).

the current sentence from left-to-right, starting with the first noun group to the right of the pronoun (for cataphora). The first noun group that agrees with the pronoun with respect to number, gender, and person is chosen as the antecedent” (Kehler et al., 2004).

Lappin and Leass (1994) was an influential entity-based system that used weights to combine syntactic and other features, extended soon after by Kennedy and Boguraev (1996) whose system avoids the need for full syntactic parses.

Approximately contemporaneously centering (Grosz et al., 1995) was applied to pronominal anaphora resolution by Brennan et al. (1987), and a wide variety of work followed focused on centering’s use in coreference (Kameyama 1986, Di Eugenio 1990, Walker et al. 1994, Di Eugenio 1996, Strube and Hahn 1996, Kehler 1997a, Tetreault 2001, Iida et al. 2003). Kehler and Rohde (2013) show how centering can be integrated with coherence-driven theories of pronoun interpretation. See Chapter 25 for the use of centering in measuring discourse coherence.

Coreference competitions as part of the US DARPA-sponsored MUC conferences provided early labeled coreference datasets (the 1995 MUC-6 and 1998 MUC-7 corpora), and set the tone for much later work, choosing to focus exclusively on the simplest cases of *identity coreference* (ignoring difficult cases like bridging, metonymy, and part-whole) and drawing the community toward supervised machine learning and metrics like the MUC metric (Vilain et al., 1995). The later ACE evaluations produced labeled coreference corpora in English, Chinese, and Arabic that were widely used for model training and evaluation.

This DARPA work influenced the community toward supervised learning beginning in the mid-90s (Connolly et al. 1994, Aone and Bennett 1995, McCarthy and Lehnert 1995). Soon et al. (2001) laid out a set of basic features, extended by Ng and Cardie (2002b), and a series of machine learning models followed over the next 15 years. These often focused separately on pronominal anaphora resolution (Kehler et al. 2004, Bergsma and Lin 2006), full NP coreference (Cardie and Wagstaff 1999, Ng and Cardie 2002b, Ng 2005a) and definite NP reference (Poesio and Vieira 1998, Vieira and Poesio 2000), as well as separate anaphoricity detection (Bean and Riloff 1999, Bean and Riloff 2004, Ng and Cardie 2002a, Ng 2004), or singleton detection (de Marneffe et al., 2015).

The move from mention-pair to mention-ranking approaches was pioneered by Yang et al. (2003) and Iida et al. (2003) who proposed pairwise ranking methods, then extended by Denis and Baldridge (2008) who proposed to do ranking via a softmax over all prior mentions. The idea of doing mention detection, anaphoricity, and coreference jointly in a single end-to-end model grew out of the early proposal of Ng (2005b) to use a dummy antecedent for mention-ranking, allowing ‘non-referential’ to be a choice for coreference classifiers, Denis and Baldridge’s 2007 joint system combining anaphoricity classifier probabilities with coreference probabilities, the Denis and Baldridge (2008) ranking model, and the Rahman and Ng (2009) proposal to train the two models jointly with a single objective.

Simple rule-based systems for coreference returned to prominence in the 2010s, partly because of their ability to encode entity-based features in a high-precision way (Zhou et al. 2004b, Haghighi and Klein 2009, Raghunathan et al. 2010, Lee et al. 2011, Lee et al. 2013, Hajishirzi et al. 2013) but in the end they suffered from an inability to deal with the semantics necessary to correctly handle cases of common noun coreference.

A return to supervised learning led to a number of advances in mention-ranking models which were also extended into neural architectures, for example using re-

inforcement learning to directly optimize coreference evaluation models [Clark and Manning \(2016a\)](#), doing end-to-end coreference all the way from span extraction ([Lee et al. 2017b](#), [Zhang et al. 2018](#)). Neural models also were designed to take advantage of global entity-level information ([Clark and Manning 2016b](#), [Wiseman et al. 2016](#), [Lee et al. 2018](#)).

Coreference is also related to the task of **entity linking** discussed in Chapter 11. Coreference can help entity linking by giving more possible surface forms to help link to the right Wikipedia page, and conversely entity linking can help improve coreference resolution. Consider this example from [Hajishirzi et al. \(2013\)](#):

(24.81) [Michael Eisner]₁ and [Donald Tsang]₂ announced the grand opening of [[Hong Kong]₃ Disneyland]₄ yesterday. [Eisner]₁ thanked [the President]₂ and welcomed [fans]₅ to [the park]₄.

Integrating entity linking into coreference can help draw encyclopedic knowledge (like the fact that *Donald Tsang* is a president) to help disambiguate the mention *the President*. [Ponzetto and Strube \(2006\)](#) 2007 and [Ratinov and Roth \(2012\)](#) showed that such attributes extracted from Wikipedia pages could be used to build richer models of entity mentions in coreference. More recent research shows how to do linking and coreference jointly ([Hajishirzi et al. 2013](#), [Zheng et al. 2013](#)) or even jointly with named entity tagging as well ([Durrett and Klein 2014](#)).

metonymy

The coreference task as we introduced it involves a simplifying assumption that the relationship between an anaphor and its antecedent is one of *identity*: the two coreferring mentions refer to the identical discourse referent. In real texts, the relationship can be more complex, where different aspects of a discourse referent can be neutralized or refocused. For example (24.82) ([Recasens et al., 2011](#)) shows an example of **metonymy**, in which the capital city *Washington* is used metonymically to refer to the US. (24.83-24.84) show other examples ([Recasens et al., 2011](#)):

(24.82) a strict interpretation of a policy requires **The U.S.** to notify foreign dictators of certain coup plots ... **Washington** rejected the bid ...

(24.83) I once crossed that border into Ashgh-Abad on Nowruz, the Persian New Year. In the South, everyone was celebrating **New Year**; to the North, **it** was a regular day.

(24.84) In France, **the president** is elected for a term of seven years, while in the United States **he** is elected for a term of four years.

For further linguistic discussions of these complications of coreference see [Pustejovsky \(1991\)](#), [van Deemter and Kibble \(2000\)](#), [Poesio et al. \(2006\)](#), [Fauconnier and Turner \(2008\)](#), [Versley \(2008\)](#), and [Barker \(2010\)](#).

[Ng \(2017\)](#) offers a useful compact history of machine learning models in coreference resolution. There are three excellent book-length surveys of anaphora/coreference resolution, covering different time periods: [Hirst \(1981\)](#) (early work until about 1981), [Mitkov \(2002\)](#) (1986-2001), and [Poesio et al. \(2016\)](#) (2001-2015).

Andy Kehler wrote the Discourse chapter for the 2000 first edition of this textbook, which we used as the starting point for the second-edition chapter, and there are some remnants of Andy's lovely prose still in this third-edition coreference chapter.

Exercises